Contents

- Modeling 1: Make a quick fit using astropy.modeling
- Modeling 2: Create a User Defined Model using astropy.modeling
- Compound models
- Custom model

This is a set of tutorials on how to use the `astropy.modeling` package to model astronomical data. We will become familiar with the models available in astropy.modeling, learn how to make a quick fit to some data, and how to define a new model using a compound model and a custom model.

Authors

Rocio Kiman, Lia Corrales, Zé Vinícius, Kelle Cruz, Stephanie T. Douglas

Learning Goals

- Use `astroquery` to download data from Vizier
- Use basic models in `astropy.modeling`
- Learn common functions to fit
- Generate a quick fit to data
- Plot the model with the data
- Compare different models and fitters

Keywords

modeling, model fitting, astrostatistics, astroquery, Vizier, scipy, matplotlib, error bars, scatter plots

Summary

In this tutorial, we will become familiar with the models available in astropy.modeling and learn how to make a quick fit to our data.

Imports

```
import numpy as np
import matplotlib.pyplot as plt
from astropy.modeling import models, fitting
from astroquery.vizier import Vizier
import scipy.optimize

# Make plots display in notebooks
%matplotlib inline
```

1) Fit a Linear model: Three steps to fit data using astropy.modeling

We are going to start with a **linear fit to real data**. The data comes from the paper Bhardwaj et al. 2017. This is a catalog of **Type II Cepheids**, which is a type of **variable stars** that pulsate with a period between 1 and 50 days. In this part of the tutorial, we are going to measure the **Cepheids Period-Luminosity** relation using `astropy.modeling`. This relation states that if a star has a longer period, the luminosity we measure is higher.

To get it, we are going to import it from Vizier using astroquery.

```
catalog = Vizier.get_catalogs("J/A+A/605/A100")
```

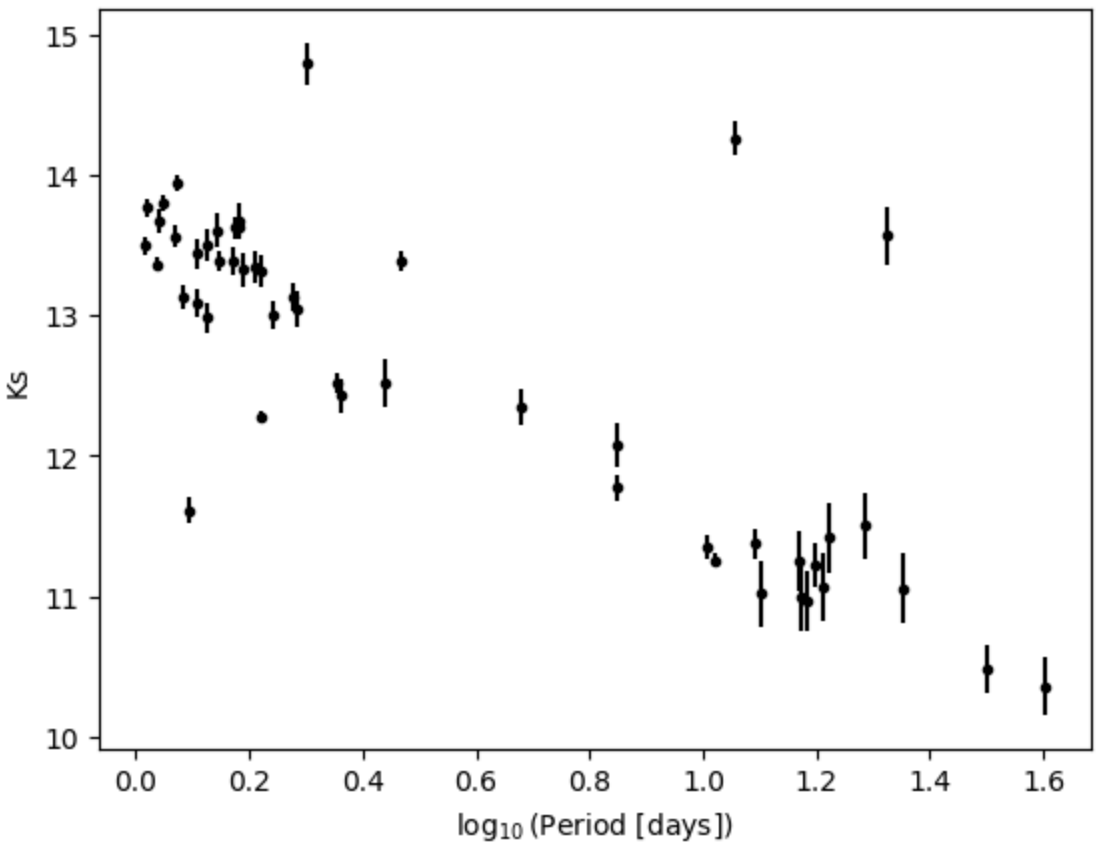
This catalog has a lot of information, but for this tutorial we are going to work only with periods and magnitudes. Let's grab them using the keywords `'Period'` and `<Ksmag>`. Note that `'e_<Ksmag>'` refers to the error bars in the magnitude measurements.

```
period = np.array(catalog[0]["Period"])
log_period = np.log10(period)
k_mag = np.array(catalog[0]["<Ksmag>"])
k_mag_err = np.array(catalog[0]["e_<Ksmag>"])
```

Let's take a look at the magnitude measurements as a function of period:

```
plt.errorbar(log_period, k_mag, k_mag_err, fmt="k.")
plt.xlabel(r"$\log_{10}$(Period [days])")
plt.ylabel("Ks")
```

```
Text(0, 0.5, 'Ks')
```



One could say that there is a linear relationship between log period and magnitudes. To probe it, we want to make a fit to the data. This is where `astropy.modeling` is useful. We are going to understand how in three simple lines we can make any fit we want. We are going to start with the linear fit, but first, let's understand what a model and a fitter are.

Models in Astropy

Models in Astropy are known parametrized functions. With this format they are easy to define and to use, given that we do not need to write the function expression every time we want to use a model, just the name. They can be linear or non-linear in the variables. Some examples of models are:

- Gaussian1D
- Trapezoid1D
- Polynomial1D
- Sine1D
- Linear1D
- The list continues.

Fitters in Astropy

Fitters in Astropy are the classes resposable for making the fit. They can be linear or non-linear in the parameters (no the variable, like models). Some examples are:

- LevMarLSQFitter() Levenberg-Marquardt algorithm and least squares statistic.
- LinearLSQFitter() A class performing a linear least square fitting.
- SLSQPLSQFitter() SLSQP optimization algorithm and least squares statistic.
- SimplexLSQFitter() Simplex algorithm and least squares statistic.
- More detailles here

Now we continue with our fitting.

Step 1: Model

First we need to choose which model we are going to use to fit to our data. As we said before, our data looks like a linear relation, so we are going to use a linear model.

```
model = models.Linear1D()
```

Step 2: Fitter

Second we are going to choose the fitter we want to use. This choice is basically which method we want to use to fit the model to the data. In this case we are going to use the Linear Least Square Fitting. In the next exercise (Modeling 2: Create a User Defined Model) we are going to analyze how to choose the fitter.

```
fitter = fitting.LinearLSQFitter()
```

Step 3: Fit Data

Finally, we give to our **fitter** (method to fit the data) the **model** and the **data** to perform the fit. Note that we are including weights: This means that values with higher error will have smaller weight (less importance) in the fit, and the contrary for data with smaller errors. This way of fitting is called *Weighted Linear Least Squares* and you can find more information about it here or here. Note that the fitting routine takes weights as 1/error and squares them for you, as indicated in the description of the function.

```
best_fit = fitter(model, log_period, k_mag, weights=1.0 / k_mag_err)
print(best_fit)
```

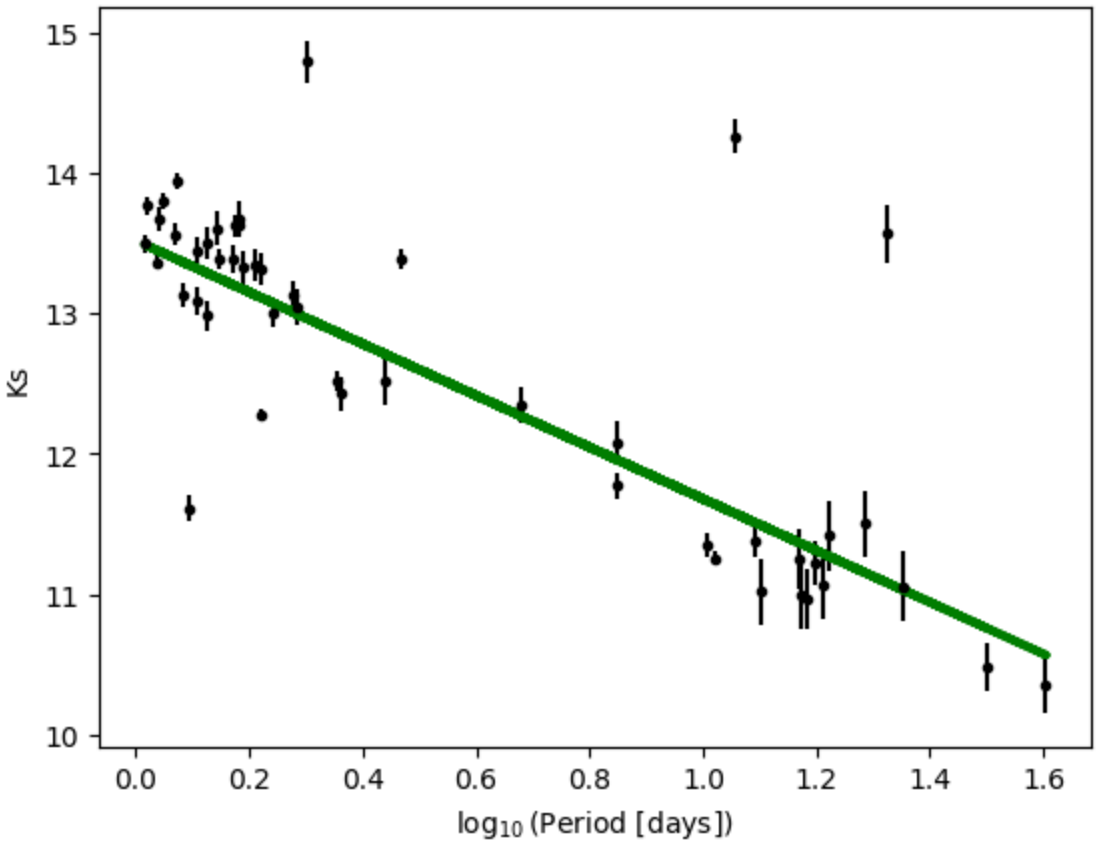
```
Model: Linear1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
           slope            intercept
    ------------------- ------------------
    -1.8388029755798254 13.517530405305104
```

And that's it!

We can evaluate the fit at our particular x axis by doing `best_fit(x)`.

```
plt.errorbar(log_period, k_mag, k_mag_err, fmt="k.")
plt.plot(log_period, best_fit(log_period), color="g", linewidth=3)
plt.xlabel(r"$\log_{10}$(Period [days])")
plt.ylabel("Ks")
```

```
Text(0, 0.5, 'Ks')
```



Conclusion: Remember, you can fit data with three lines of code:

1. Choose a model.
2. Choose a fitter.
3. Pass to the fitter the model and the data to perform fit.

Exercise

Use the model `Polynomial1D(degree=1)` to fit the same data and compare the results.

2) Fit a Polynomial model: Choose fitter wisely

For our second example, let's fit a polynomial of degree more than 1. In this case, we are going to create fake data to make the fit. Note that we're adding gaussian noise to the data with the function

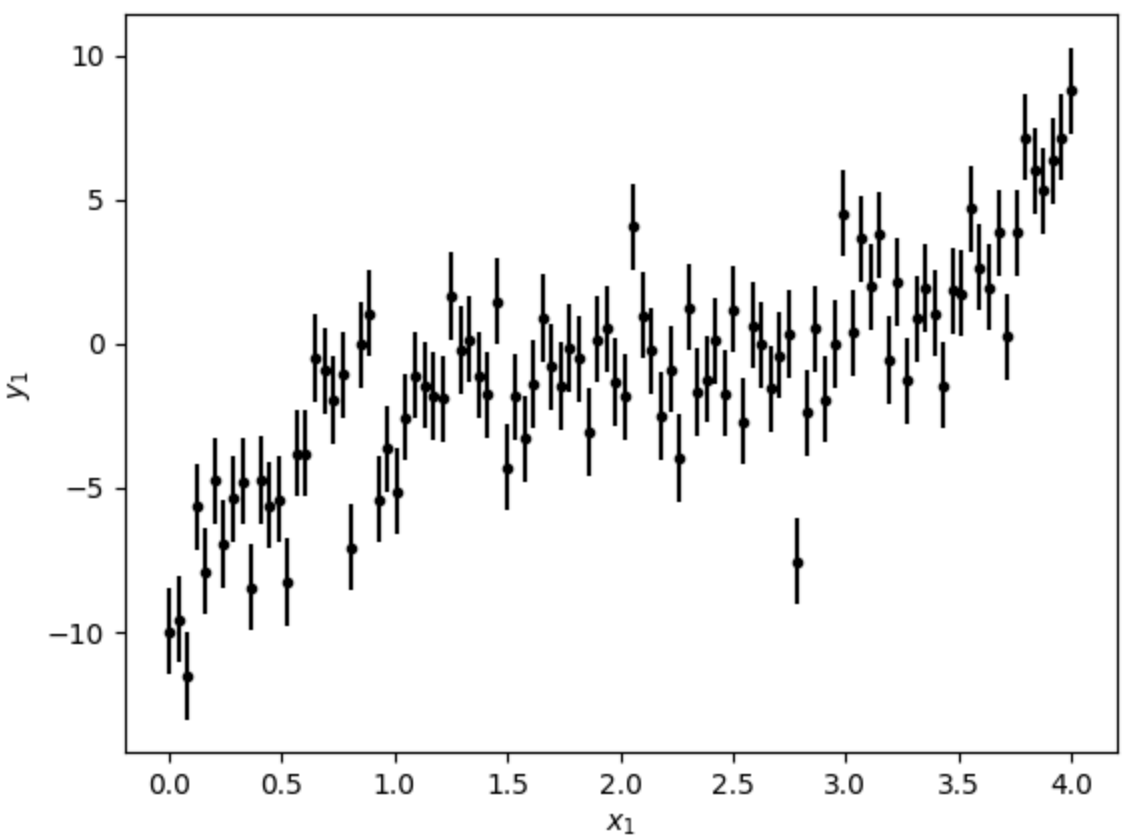
`np.random.normal(0,2)` which gives a random number from a gaussian distribution with mean 0 and standard deviation 2.

```
N = 100
x1 = np.linspace(0, 4, N)  # Makes an array from 0 to 4 of N elements
y1 = x1**3 - 6 * x1**2 + 12 * x1 - 9
# Now we add some noise to the data
y1 += np.random.normal(0, 2, size=len(y1))  # One way to add random gaussian noise
sigma = 1.5
y1_err = np.ones(N) * sigma
```

Let's plot it to see how it looks:

```
plt.errorbar(x1, y1, yerr=y1_err, fmt="k.")
plt.xlabel("$x_1$")
plt.ylabel("$y_1$")
```

```
Text(0, 0.5, '$y_1$')
```



To fit this data let's remember the three steps: model, fitter and perform fit.

```
model_poly = models.Polynomial1D(degree=3)
fitter_poly = fitting.LinearLSQFitter()
best_fit_poly = fitter_poly(model_poly, x1, y1, weights=1.0 / y1_err)
```

```
print(best_fit_poly)
```

```
Model: Polynomial1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Degree: 3
Parameters:
            c0                  c1                  c2                  c3
    ------------------ ------------------ ------------------ ------------------
    -9.766441536185068 13.520683626824438 -6.745723767810022 1.1061382212238513
```

What would happend if we use a different fitter (method)? Let's use the same model but with `SimplexLSQFitter` as fitter.

```
fitter_poly_2 = fitting.SimplexLSQFitter()
best_fit_poly_2 = fitter_poly_2(model_poly, x1, y1, weights=1.0 / y1_err)
```

```
WARNING: Model is linear in parameters; consider using linear fitting methods. [astr
WARNING: The fit may be unsuccessful; Maximum number of iterations reached. [astropy
```

```
print(best_fit_poly_2)
```

```
Model: Polynomial1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Degree: 3
Parameters:
             c0                   c1                   c2                   c3
    -------------------- ------------------- ------------------- -------------------
    0.017124530973053394 -0.6827981656238378 -0.7583618188337732 0.33505214184979437
```

Note that we got a warning after using `SimplexLSQFitter` to fit the data. The first line says:

`WARNING: Model is linear in parameters; consider using linear fitting methods. [astropy.modeling.fitting]`

If we look at the model we chose: $y = c_0 + c_1 \times x + c_2 \times x^2 + c_3 \times x^3$, it is linear in the parameters c_i. The warning means that `SimplexLSQFitter` works better with models that are not linear in the parameters, and that we should use a linear fitter like `LinearLSQFitter`. The second line says:

`WARNING: The fit may be unsuccessful; Maximum number of iterations reached. [astropy.modeling.optimizers]`

So it's not surprising that the results are different, because this means that the fitter is not working properly. Let's discuss a method of choosing between fits and remember to **pay attention** when you choose the **fitter**.

Compare results

One way to check which model parameters are a better fit is calculating the [Reduced Chi Square Value](). Let's define a function to do that because we're going to use it several times.

```
def calc_reduced_chi_square(fit, x, y, yerr, N, n_free):
    """
    fit (array) values for the fit
    x,y,yerr (arrays) data
    N total number of points
    n_free number of parameters we are fitting
    """
    return 1.0 / (N - n_free) * sum(((fit - y) / yerr) ** 2)
```

```
reduced_chi_squared = calc_reduced_chi_square(best_fit_poly(x1), x1, y1, y1_err, N,
print("Reduced Chi Squared with LinearLSQFitter: {}".format(reduced_chi_squared))
```

```
Reduced Chi Squared with LinearLSQFitter: 1.8152214211896238
```

```
reduced_chi_squared = calc_reduced_chi_square(best_fit_poly_2(x1), x1, y1, y1_err, N
print("Reduced Chi Squared with SimplexLSQFitter: {}".format(reduced_chi_squared))
```

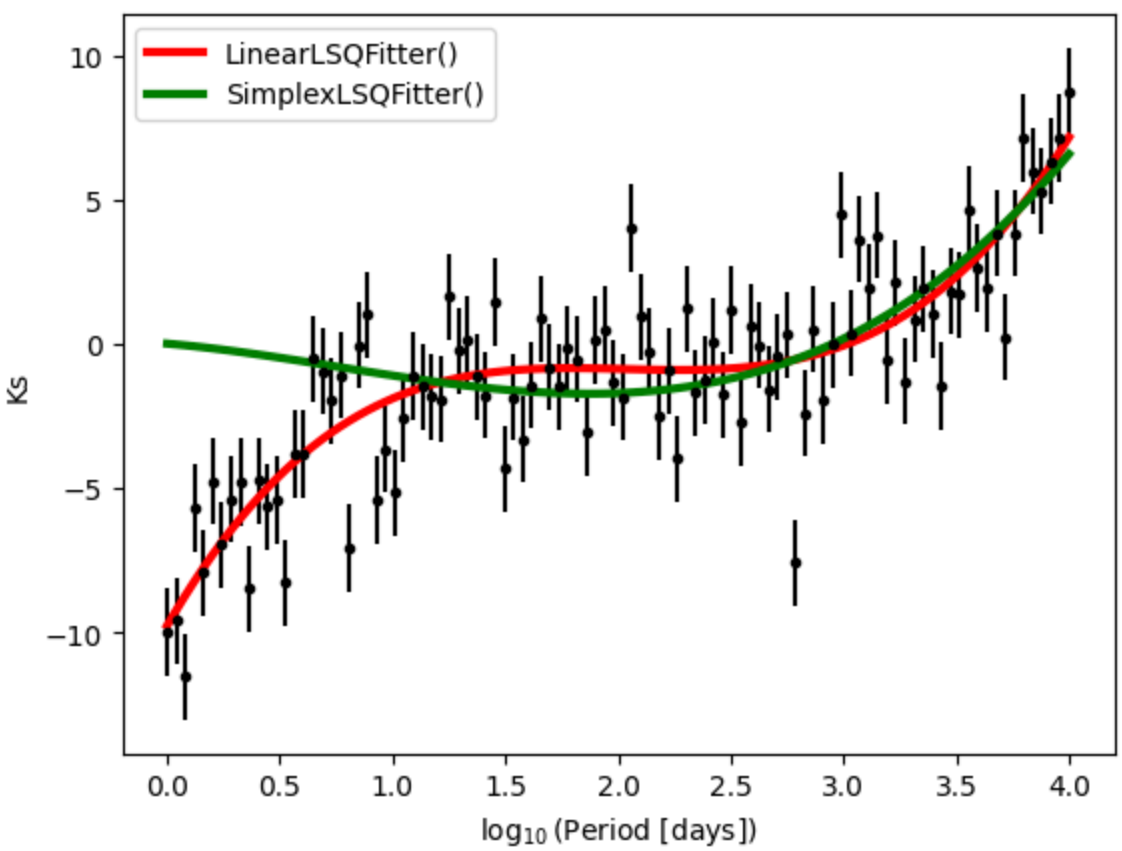
```
Reduced Chi Squared with SimplexLSQFitter: 5.207094663656951
```

As we can see, the *Reduced Chi Square* for the first fit is closer to one, which means this fit is better. Note that this is what we expected after the discussion of the warnings.

We can also compare the two fits visually:

```
plt.errorbar(x1, y1, yerr=y1_err, fmt="k.")
plt.plot(x1, best_fit_poly(x1), color="r", linewidth=3, label="LinearLSQFitter()")
plt.plot(x1, best_fit_poly_2(x1), color="g", linewidth=3, label="SimplexLSQFitter()
plt.xlabel(r"$\log_{10}$(Period [days])")
plt.ylabel("Ks")
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fccdd257a10>
```



Results are as espected, the fit performed with the linear fitter is better than the second, non linear one.

Conclusion: Pay attention when you choose the fitter.

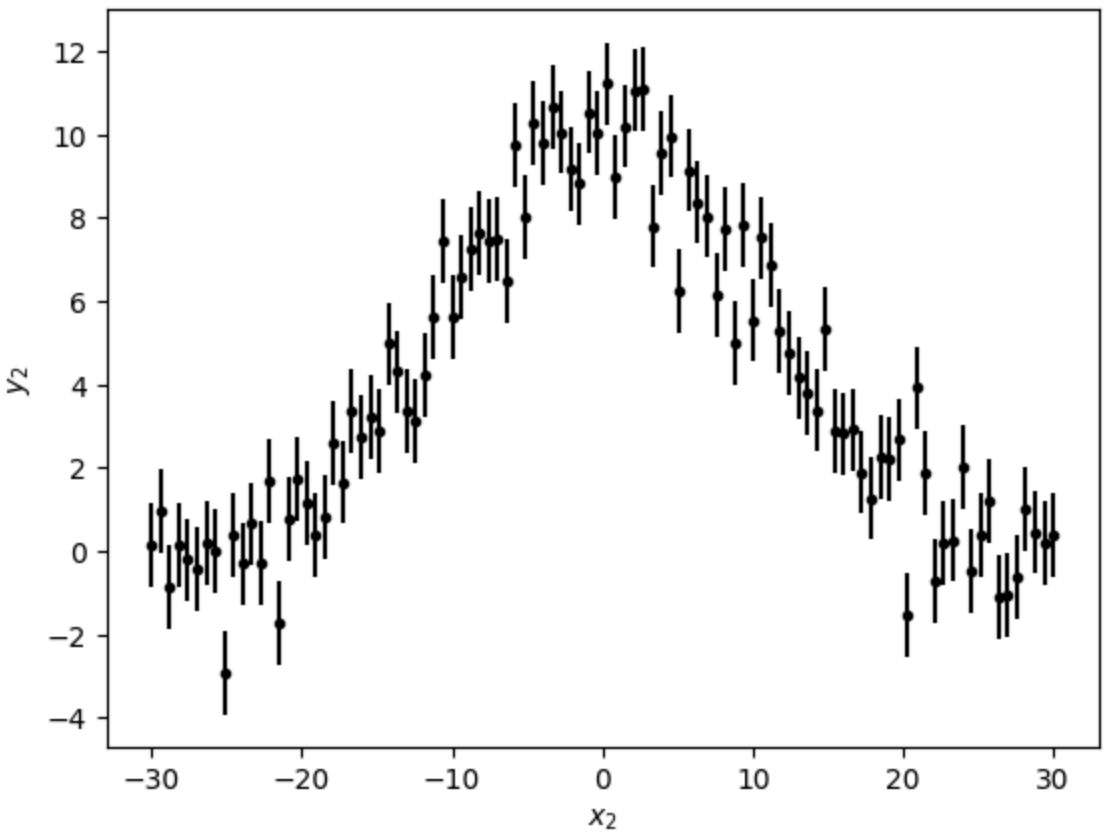
3) Fit a Gaussian: Let’s compare to scipy

Scipy has the function scipy.optimize.curve_fit to fit in a similar way that we are doing. Let’s compare the two methods with fake data in the shape of a Gaussian.

```
mu, sigma, amplitude = 0.0, 10.0, 10.0
N2 = 100
x2 = np.linspace(-30, 30, N)
y2 = amplitude * np.exp(-((x2 - mu) ** 2) / (2 * sigma**2))
y2 = np.array(
    [y_point + np.random.normal(0, 1) for y_point in y2]
)  # Another way to add random gaussian noise
sigma = 1
y2_err = np.ones(N) * sigma
```

```
plt.errorbar(x2, y2, yerr=y2_err, fmt="k.")
plt.xlabel("$x_2$")
plt.ylabel("$y_2$")
```

```
Text(0, 0.5, '$y_2$')
```



Let's do our three steps to make the fit we want. For this fit we're going to use a non-linear fitter, `LevMarLSQFitter`, because the model we need (`Gaussian1D`) is non-linear in the parameters.

```
model_gauss = models.Gaussian1D()
fitter_gauss = fitting.LevMarLSQFitter()
best_fit_gauss = fitter_gauss(model_gauss, x2, y2, weights=1 / y2_err)
```

```
print(best_fit_gauss)
```

```
Model: Gaussian1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
        amplitude             mean             stddev
    ------------------ ------------------ ----------------
    10.204543320873979 0.1919178735584224 9.85164655199492
```

We can get the covariance matrix from `LevMarLSQFitter`, which provides an error for our fit parameters by doing `fitter.fit_info['param_cov']`. The elements in the diagonal of this matrix are the square of the errors. We can check the order of the parameters using:

```
model_gauss.param_names
```

```
('amplitude', 'mean', 'stddev')
```

```
cov_diag = np.diag(fitter_gauss.fit_info["param_cov"])
print(cov_diag)
```

```
[0.05210156 0.06471393 0.06487102]
```

Then:

```
print(
    "Amplitude: {} +\- {}".format(best_fit_gauss.amplitude.value, np.sqrt(cov_diag[0
)
print("Mean: {} +\- {}".format(best_fit_gauss.mean.value, np.sqrt(cov_diag[1])))
print(
    "Standard Deviation: {} +\- {}".format(
        best_fit_gauss.stddev.value, np.sqrt(cov_diag[2])
    )
)
```

```
Amplitude: 10.204543320873979 +\- 0.22825766424549349
Mean: 0.1919178735584224 +\- 0.2543893224573705
Standard Deviation: 9.85164655199492 +\- 0.25469789812030813
```

```
<>:2: SyntaxWarning: invalid escape sequence '\-'
<>:4: SyntaxWarning: invalid escape sequence '\-'
<>:6: SyntaxWarning: invalid escape sequence '\-'
<>:2: SyntaxWarning: invalid escape sequence '\-'
<>:4: SyntaxWarning: invalid escape sequence '\-'
<>:6: SyntaxWarning: invalid escape sequence '\-'
/tmp/ipykernel_3552/1659586526.py:2: SyntaxWarning: invalid escape sequence '\-'
  "Amplitude: {} +\- {}".format(best_fit_gauss.amplitude.value, np.sqrt(cov_diag[0]))
/tmp/ipykernel_3552/1659586526.py:4: SyntaxWarning: invalid escape sequence '\-'
  print("Mean: {} +\- {}".format(best_fit_gauss.mean.value, np.sqrt(cov_diag[1])))
/tmp/ipykernel_3552/1659586526.py:6: SyntaxWarning: invalid escape sequence '\-'
  "Standard Deviation: {} +\- {}".format(
```

We can apply the same method with `scipy.optimize.curve_fit`, and compare the results using again the *Reduced Chi Square Value*.

```
def f(x, a, b, c):
    return a * np.exp(-((x - b) ** 2) / (2.0 * c**2))
```

```
p_opt, p_cov = scipy.optimize.curve_fit(f, x2, y2, sigma=y1_err)
a, b, c = p_opt
best_fit_gauss_2 = f(x2, a, b, c)
```

```
print(p_opt)
```

```
[10.20454451  0.19191914  9.85164425]
```

```
print("Amplitude: {} +\- {}".format(p_opt[0], np.sqrt(p_cov[0, 0])))
print("Mean: {} +\- {}".format(p_opt[1], np.sqrt(p_cov[1, 1])))
print("Standard Deviation: {} +\- {}".format(p_opt[2], np.sqrt(p_cov[2, 2])))
```

```
Amplitude: 10.204544513903032 +\- 0.24558628989943265
Mean: 0.19191914251073947 +\- 0.2737028472248566
Standard Deviation: 9.851644250688773 +\- 0.2740348254105184
```

```
<>:1: SyntaxWarning: invalid escape sequence '\-'
<>:2: SyntaxWarning: invalid escape sequence '\-'
<>:3: SyntaxWarning: invalid escape sequence '\-'
<>:1: SyntaxWarning: invalid escape sequence '\-'
<>:2: SyntaxWarning: invalid escape sequence '\-'
<>:3: SyntaxWarning: invalid escape sequence '\-'
/tmp/ipykernel_3552/2222327920.py:1: SyntaxWarning: invalid escape sequence '\-'
  print("Amplitude: {} +\- {}".format(p_opt[0], np.sqrt(p_cov[0, 0])))
/tmp/ipykernel_3552/2222327920.py:2: SyntaxWarning: invalid escape sequence '\-'
  print("Mean: {} +\- {}".format(p_opt[1], np.sqrt(p_cov[1, 1])))
/tmp/ipykernel_3552/2222327920.py:3: SyntaxWarning: invalid escape sequence '\-'
  print("Standard Deviation: {} +\- {}".format(p_opt[2], np.sqrt(p_cov[2, 2])))
```

Compare results

```
reduced_chi_squared = calc_reduced_chi_square(best_fit_gauss(x2), x2, y2, y2_err, N2
print("Reduced Chi Squared using astropy.modeling: {}".format(reduced_chi_squared))
```

```
Reduced Chi Squared using astropy.modeling: 1.1576001525793589
```

```
reduced_chi_squared = calc_reduced_chi_square(best_fit_gauss_2, x2, y2, y2_err, N2,
print("Reduced Chi Squared using scipy: {}".format(reduced_chi_squared))
```

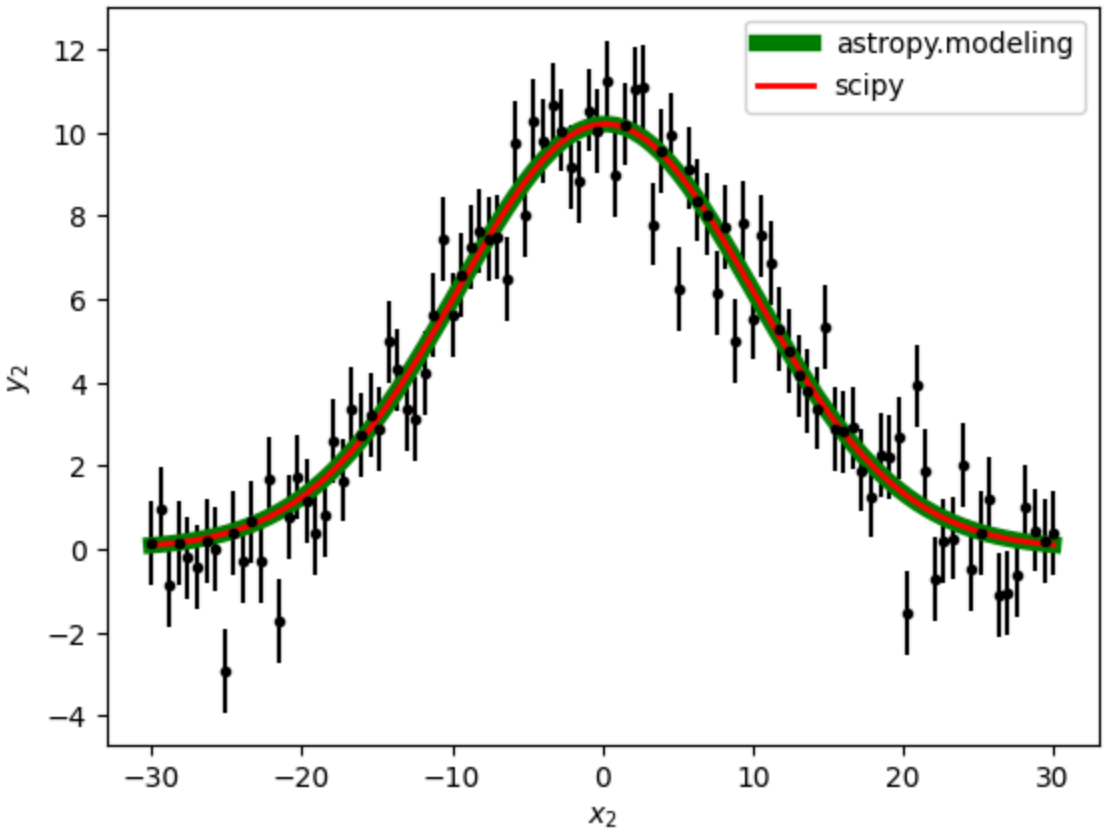
```
Reduced Chi Squared using scipy: 1.1576001525612583
```

As we can see there is a very small difference in the *Reduced Chi Squared*. This actually needed to happen, because the fitter in `astropy.modeling` uses scipy to fit. The advantage of using `astropy.modeling` is you only need to change the name of the fitter and the model to perform a

completely different fit, while scipy require us to remember the expression of the function we wanted to use.

```
plt.errorbar(x2, y2, yerr=y2_err, fmt="k.")
plt.plot(x2, best_fit_gauss(x2), "g-", linewidth=6, label="astropy.modeling")
plt.plot(x2, best_fit_gauss_2, "r-", linewidth=2, label="scipy")
plt.xlabel("$x_2$")
plt.ylabel("$y_2$")
plt.legend()
```

```
<matplotlib.legend.Legend at 0x7fccdcf84e90>
```



Conclusion: Choose the method most convenient for every case you need to fit. We recomend `astropy.modeling` because is easier to write the name of the function you want to fit than to remember the expression every time we want to use it. Also, `astropy.modeling` becomes useful with more complicated models like two gaussians plus a black body, but that is another tutorial.

Summary:

Let's review the conclusion we got in this tutorial:

1. You can fit data with **three lines of code**:
    - model
    - fitter
    - perform fit to data
2. **Pay attention** when you choose the **fitter**.
3. Choose the method most convenient for every case you need to fit. We recomend `astropy.modeling` to make **quick fits of known functions**.

4) Exercise: Your turn to choose

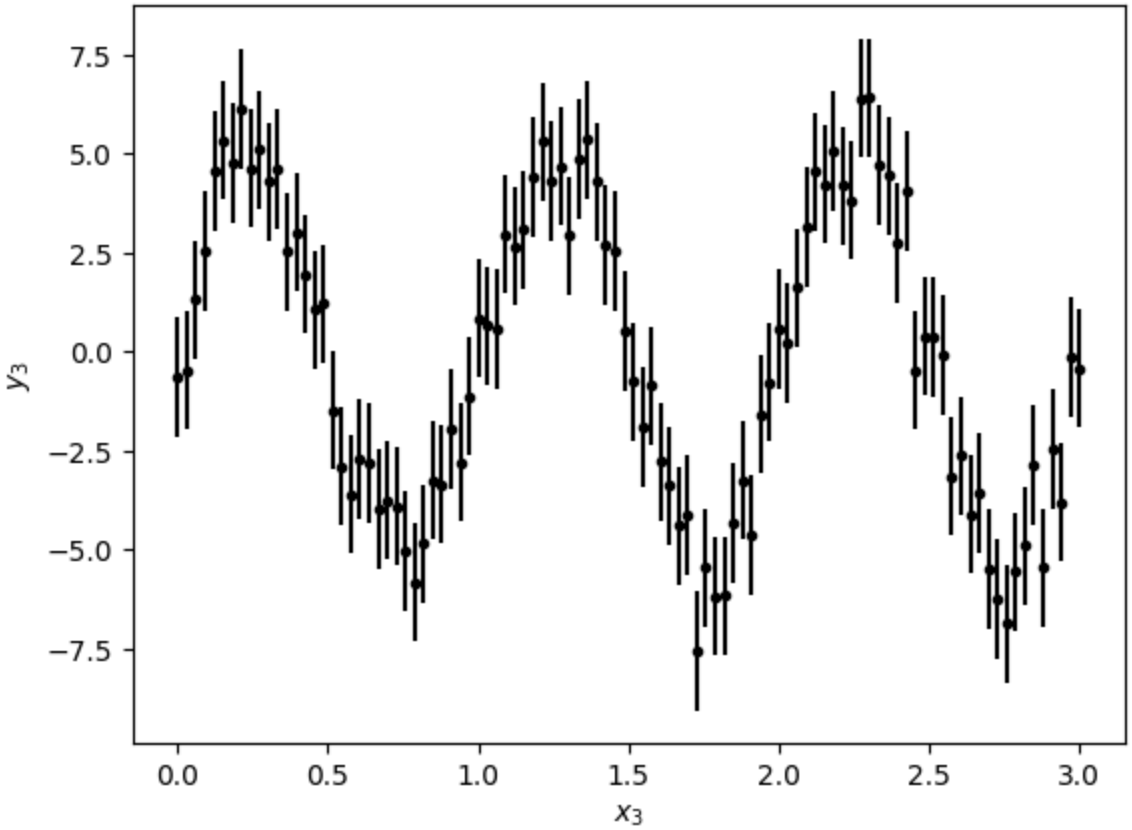
For the next data:

- Choose model and fitter to fit this data
- Compare different options

```
N3 = 100
x3 = np.linspace(0, 3, N3)
y3 = 5.0 * np.sin(2 * np.pi * x3)
y3 = np.array([y_point + np.random.normal(0, 1) for y_point in y3])
sigma = 1.5
y3_err = np.ones(N) * sigma
```

```
plt.errorbar(x3, y3, yerr=y3_err, fmt="k.")
plt.xlabel("$x_3$")
plt.ylabel("$y_3$")
```

```
Text(0, 0.5, '$y_3$')
```



Authors

Rocio Kiman, Lia Corrales, Zé Vinícius, Stephanie T. Douglas

Learning Goals

- Define a new model with `astropy`
- Identify cases were a user-defined model could be useful
- Define models in two different ways:
  - Compound models
  - Custom models

This tutorial assumes the student knows how to fit data using `astropy.modeling`. This topic is covered in the [Models-Quick-Fit tutorial](#).

Keywords

modeling, FITS, astrostatistics, matplotlib, model fitting, error bars, scatter plots

Summary

In this tutorial, we will learn how to define a new model in two ways: with a compound model and with a custom model.

Note: This tutorial assumes you have already gone through Modeling 1, which provides an introduction to `astropy.modeling`

Imports

```
import numpy as np
import matplotlib.pyplot as plt
from astropy.modeling import models, fitting
from astropy.modeling.models import custom_model
from astropy.modeling import Fittable1DModel, Parameter
from astroquery.sdss import SDSS
```

Fit an emission line in a stellar spectrum

M dwarfs are low mass stars (less than half of the mass of the sun). Currently we do not understand completely the physics inside low mass stars because they do not behave the same way higher mass stars do. For example, they stay magnetically active longer than higher mass stars. One way to measure magnetic activity is the height of the $H\alpha$ emission line. It is located at 6563 Angstroms at the spectrum.

Let's search for a spectrum of an M dwarf in the Sloan Digital Sky Survey (SDSS). First, we are going to look for the spectrum in the SDSS database. SDSS has a particular way to identify the stars it observes: it uses three numbers: Plate, Fiber and MJD (Modified Julian Date). The star we are going to use has:

- Plate: 1349
- Fiber: 216
- MJD: 52797

So go ahead, put this numbers in the website and click on Plot to visualize the spectrum. Try to localize the $H\alpha$ line.

We could download the spectrum by hand from this website, but we are going to import it using the SDSSClass from `astroquery.sdss`. We can get the spectrum using the plate, fiber and mjd in the following way:

```
spectrum = SDSS.get_spectra(plate=1349, fiberID=216, mjd=52797)[0]
```

Now that we have the spectrum…

One way to check what is inside the fits file `spectrum` is the following:

```
spectrum[1].columns
```

```
ColDefs(
    name = 'flux'; format = 'E'
    name = 'loglam'; format = 'E'
    name = 'ivar'; format = 'E'
    name = 'and_mask'; format = 'J'
    name = 'or_mask'; format = 'J'
    name = 'wdisp'; format = 'E'
    name = 'sky'; format = 'E'
    name = 'model'; format = 'E'
)
```

To plot the spectrum we need the flux as a function of wavelength (usually called lambda or λ). Note that the wavelength is in log scale: loglam, so we calculate 10^{λ} to remove this scale.

```
flux = spectrum[1].data["flux"]
lam = 10 ** (spectrum[1].data["loglam"])
```

To find the units for flux and wavelength, we look in `fitsfile[0].header`.

FITS standard requires that the header keyword ‘bunit’ or ‘BUNIT’ contains the physical units of the array values. That’s where we’ll find the flux units.

```
# Units of the flux
units_flux = spectrum[0].header["bunit"]
print(units_flux)
```

```
1E-17 erg/cm^2/s/Ang
```

Different sources will definite wavelength information differently, so we need to check the documentation. For example, this SDSS tutorial tells us what header keyword to look at.

```
# Units of the wavelegth
units_wavelength_full = spectrum[0].header["WAT1_001"]
print(units_wavelength_full)
```

```
wtype=linear label=Wavelength units=Angstroms
```

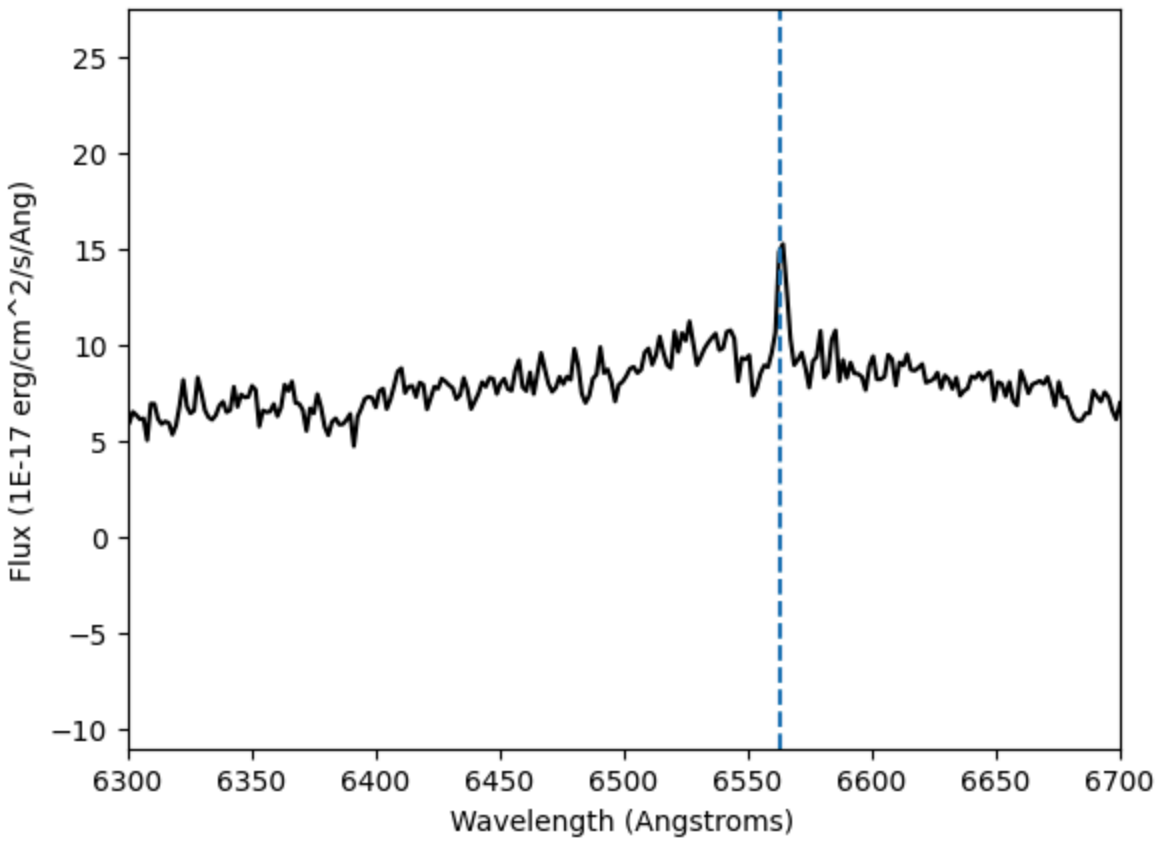
We are going to select only the characters of the unit we care about: Angstroms

```
units_wavelength = units_wavelength_full[36:]
print(units_wavelength)
```

```
Angstroms
```

Now we are ready to plot the spectrum with all the information.

```
plt.plot(lam, flux, color="k")
plt.xlim(6300, 6700)
plt.axvline(x=6563, linestyle="--")
plt.xlabel("Wavelength ({})".format(units_wavelength))
plt.ylabel("Flux ({})".format(units_flux))
plt.show()
```



We just plotted our spectrum! Check different ranges of wavelength to see how the full spectrum looks like in comparison to the one we saw before.

Fit an Emission Line with a Gaussian Model

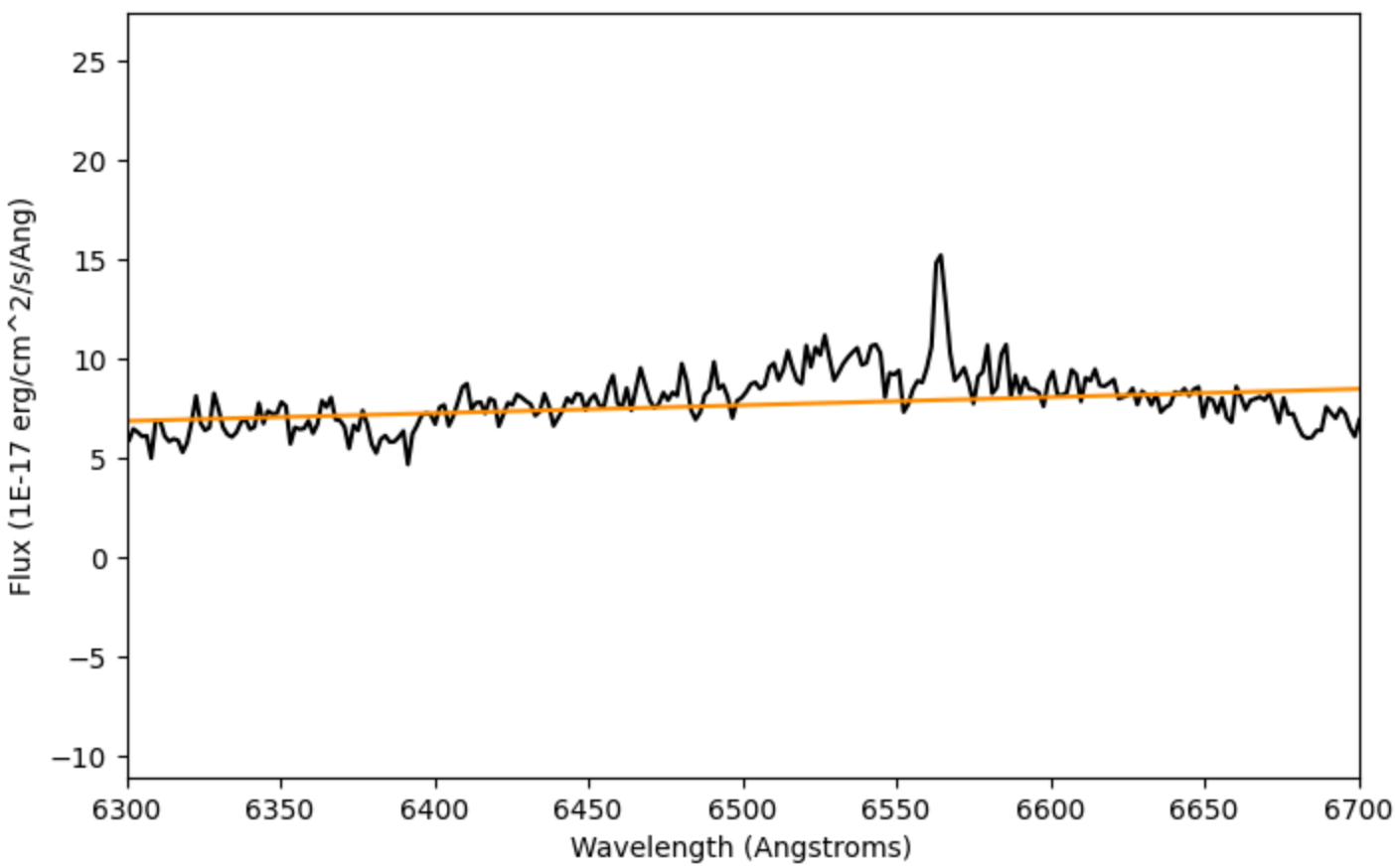
The blue dashed line marks the $H\alpha$ emission line. We can tell this is an active star because it has a strong emission line.

Now, we would like to measure the height of this line. Let's use `astropy.modeling` to fit a gaussian to the $H\alpha$ line. We are going to initialize a gaussian model at the position of the $H\alpha$ line. The idea is that the gaussian amplitude will tell us the height of the line.

```
gausian_model = models.Gaussian1D(1, 6563, 10)
fitter = fitting.LevMarLSQFitter()
gaussian_fit = fitter(gausian_model, lam, flux)
```

Let's plot the results.

```
plt.figure(figsize=(8, 5))
plt.plot(lam, flux, color="k")
plt.plot(lam, gaussian_fit(lam), color="darkorange")
plt.xlim(6300, 6700)
plt.xlabel("Wavelength (Angstroms)")
plt.ylabel("Flux ({})".format(units_flux))
plt.show()
```



We can see the fit is not doing a good job. Let's print the parameters of this fit:

```
print(gaussian_fit)
```

```
Model: Gaussian1D
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
        amplitude             mean             stddev
    ------------------ ----------------- -----------------
    16.750706286489784 9456.749529382625 2368.395705667002
```

Exercise

Go back to the previous plot and try to make the fit work. Note: **Do not spend more than 10 minutes** in this exercise. A couple of ideas to try:

- Is it not working because of the model we chose to fit? You can find more models to use here.
- Is it not working because of the fitter we chose?
- Is it not working because of the range of data we are fitting?
- Is it not working because how we are plotting the data?

One model is not enough to make this fit work. We need to combine a couple of models to make a compound model in `astropy`. The idea is that we can add, divide or multiply models that already exist in astropy.modeling and fit the compound model to our data.

For our problem we are going to combine the gaussian with a polynomial of degree 1 to account for the background spectrum close to the $H\alpha$ line. Take a look at the plot we made before to convince yourself that this is the case.

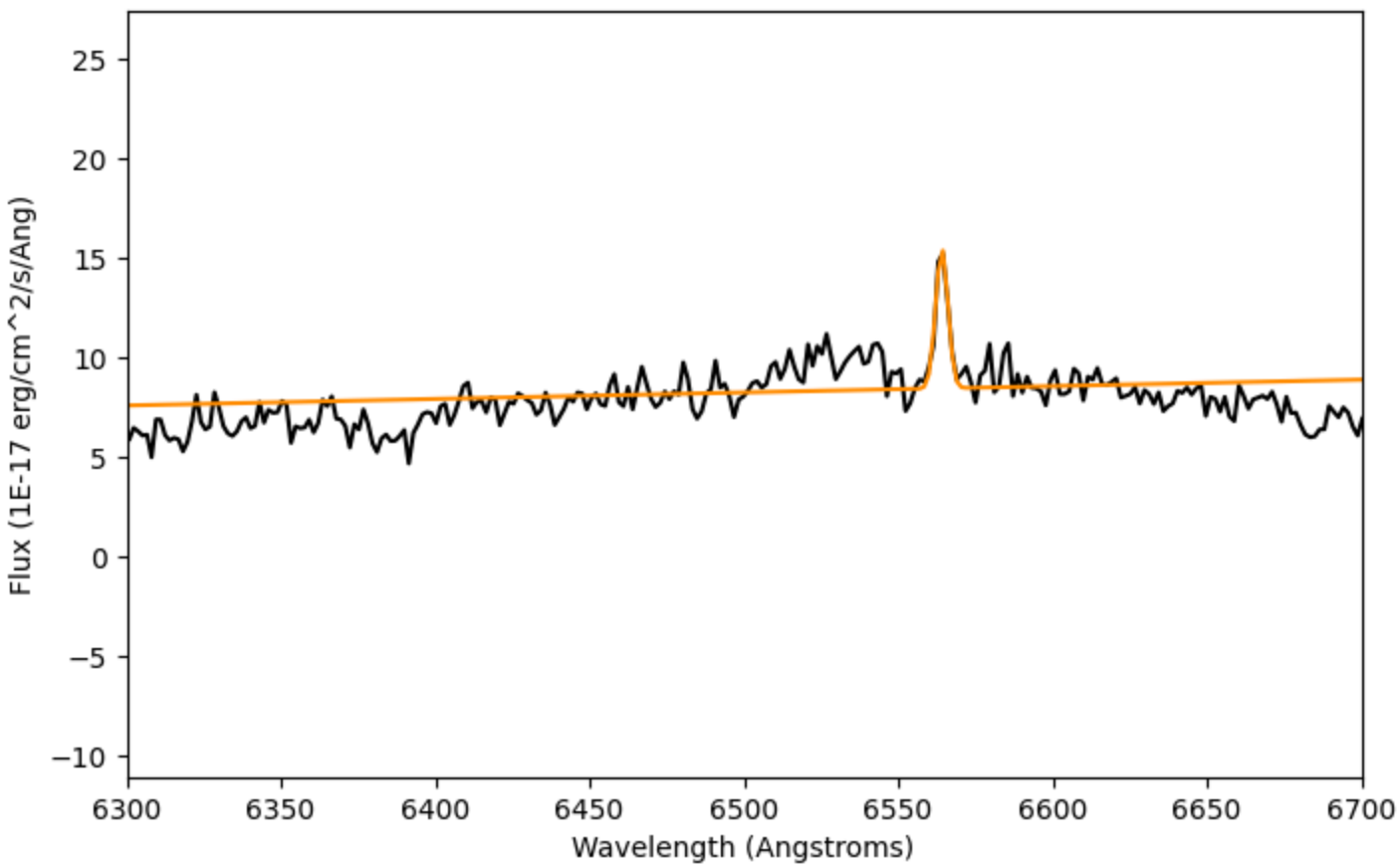
Now let's make our compound model!

```
compound_model = models.Gaussian1D(1, 6563, 10) + models.Polynomial1D(degree=1)
```

After this point, we fit the data in exactly the same way as before, except we use a compound model instead of the gaussian model.

```
fitter = fitting.LevMarLSQFitter()
compound_fit = fitter(compound_model, lam, flux)
```

```
plt.figure(figsize=(8, 5))
plt.plot(lam, flux, color="k")
plt.plot(lam, compound_fit(lam), color="darkorange")
plt.xlim(6300, 6700)
plt.xlabel("Wavelength (Angstroms)")
plt.ylabel("Flux ({})".format(units_flux))
plt.show()
```



It works! Let's take a look to the fit we just made.

```
print(compound_fit)
```

```
Model: CompoundModel
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Expression: [0] + [1]
Components:
    [0]: <Gaussian1D(amplitude=7.02171248, mean=6564.13272082, stddev=1.97715531)>

    [1]: <Polynomial1D(1, c0=-12.79335637, c1=0.00323995)>
Parameters:
       amplitude_0          mean_0      ...         c0_1                c1_1
    ---------------- ---------------- ... ------------------- --------------------
    7.021712482355418 6564.132720819791 ... -12.793356369422343 0.00323995219686687
```

Let's print all the parameters in a fancy way:

```
for x, y in zip(compound_fit.param_names, compound_fit.parameters):
    print(x, y)
```

```
amplitude_0 7.021712482355418
mean_0 6564.132720819791
stddev_0 1.977155311367857
c0_1 -12.793356369422343
c1_1 0.003239952196866877
```

We can see that the result includes all the fit parameters from the gaussian (mean, std and amplitude) and the two coefficients from the polynomial of degree 1. So now if we want to see just the amplitude:

```
compound_fit.amplitude_0
```

```
Parameter('amplitude', value=7.021712482355418)
```

Conclusions: What was the difference between the first simple Gaussian and the compound model? The linear model that we added up to the gaussian model allowed the base of the Gaussian fit to have a slope and a background level. Normal Gaussians go to zero at $\pm\inf$; this one doesn't.

Fixed or bounded model parameters

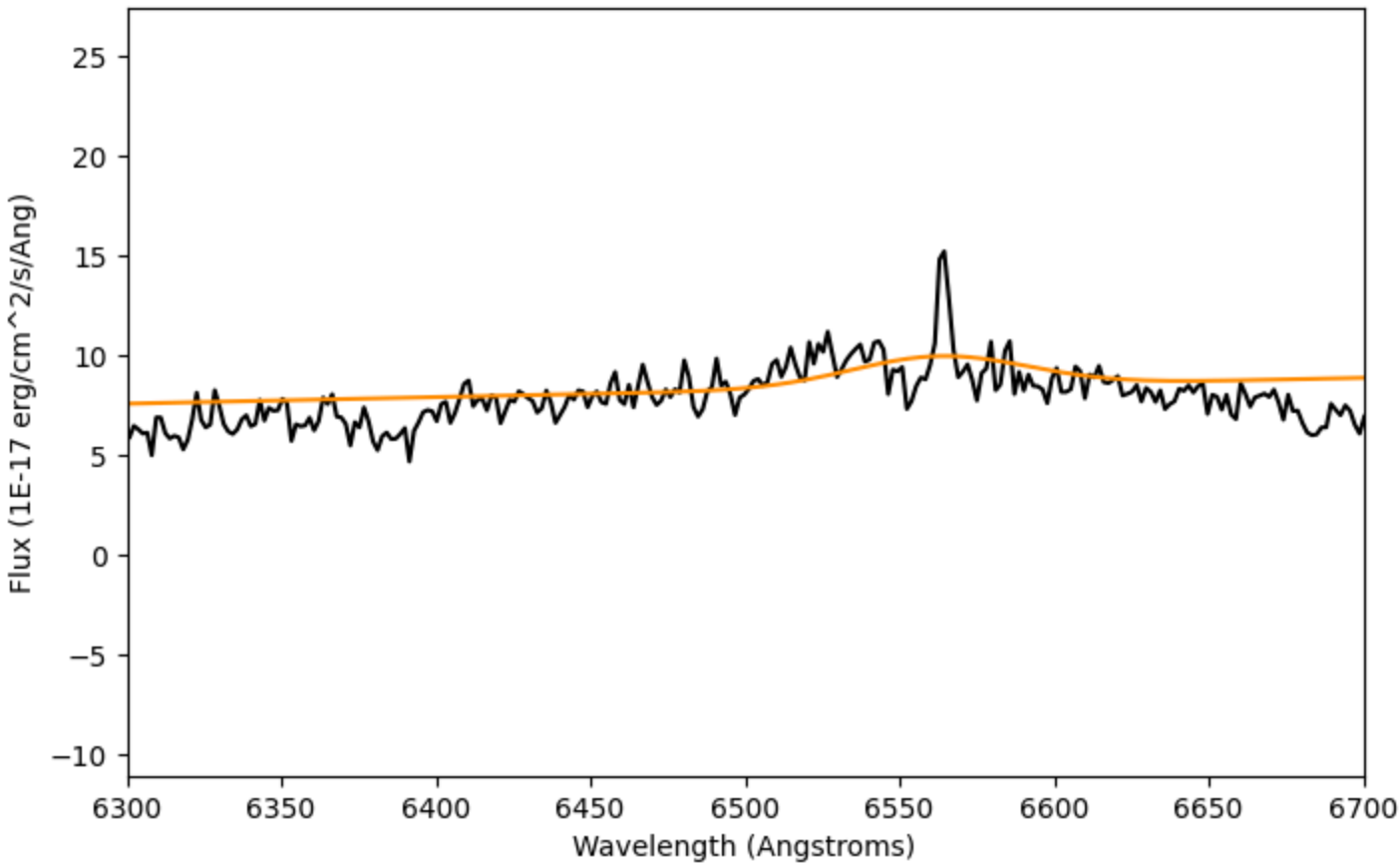
The mean value of the gaussian from our previous model indicates where the $H\alpha$ line is. In our fit result, we can tell that it is a little off from 6563 Angstroms. One way to fix this is to fix some of the parameters of the model. In `astropy.modeling` these are called **fixed parameters**.

```
compound_model_fixed = models.Gaussian1D(1, 6563, 10) + models.Polynomial1D(degree=1
compound_model_fixed.mean_0.fixed = True
```

Now let's use this new model with a fixed parameter to fit the data the same way we did before.

```
fitter = fitting.LevMarLSQFitter()
compound_fit_fixed = fitter(compound_model_fixed, lam, flux)
```

```
plt.figure(figsize=(8, 5))
plt.plot(lam, flux, color="k")
plt.plot(lam, compound_fit_fixed(lam), color="darkorange")
plt.xlim(6300, 6700)
plt.xlabel("Wavelength (Angstroms)")
plt.ylabel("Flux ({})".format(units_flux))
plt.show()
```



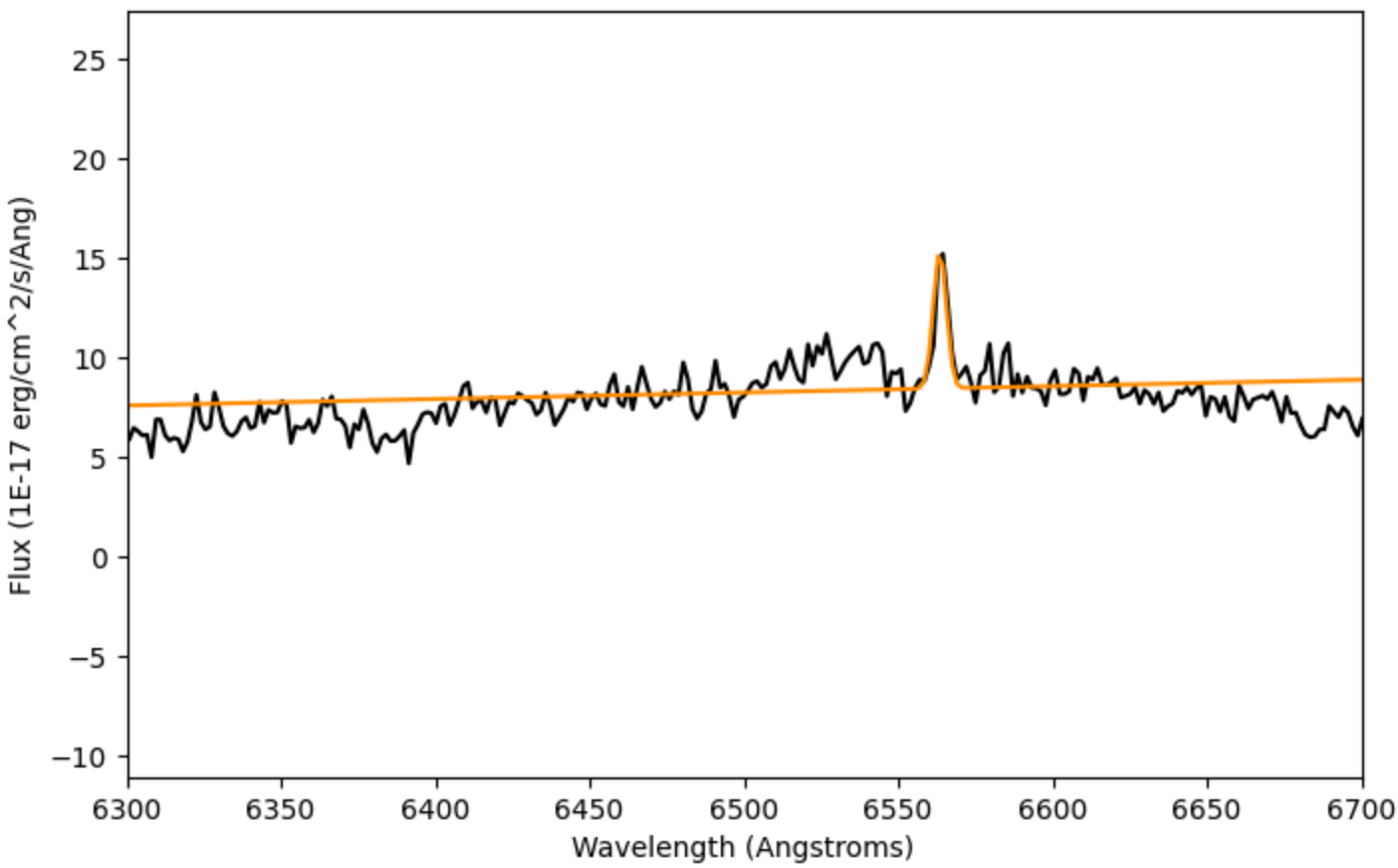

```
print(compound_fit_fixed)
```

```
Model: CompoundModel
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Expression: [0] + [1]
Components:
    [0]: <Gaussian1D(amplitude=1.53255901, mean=6563., stddev=28.57906909)>

    [1]: <Polynomial1D(1, c0=-12.79102997, c1=0.00323745)>
Parameters:
       amplitude_0     mean_0 ...         c0_1                c1_1
    ------------------ ------ ... ------------------- --------------------
    1.5325590144558827 6563.0 ... -12.791029967444068 0.003237448589619317
```

We can see in the plot that the height of the fit does not match the $H\alpha$ line height. What happend here is that we were too strict with the mean value, so we did not get a good fit. But the mean value is where we want it! Let's loosen this condition a little. Another thing we can do is to define a **minimum and maximum value** for the mean.

```
compound_model_bounded = models.Gaussian1D(1, 6563, 10) + models.Polynomial1D(degree
delta = 0.5
compound_model_bounded.mean_0.max = 6563 + delta
compound_model_bounded.mean_0.min = 6563 - delta

fitter = fitting.LevMarLSQFitter()
compound_fit_bounded = fitter(compound_model_bounded, lam, flux)
```

```
plt.figure(figsize=(8, 5))
plt.plot(lam, flux, color="k")
plt.plot(lam, compound_fit_bounded(lam), color="darkorange")
plt.xlim(6300, 6700)
plt.xlabel("Wavelength (Angstroms)")
plt.ylabel("Flux ({})".format(units_flux))
plt.show()
```




```python
print(compound_fit_bounded)
```

```
Model: CompoundModel
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Expression: [0] + [1]
Components:
    [0]: <Gaussian1D(amplitude=6.86741863, mean=6563.5, stddev=2.12272132)>

    [1]: <Polynomial1D(1, c0=-12.79313621, c1=0.00323986)>
Parameters:
      amplitude_0   mean_0 ...         c0_1                c1_1
    --------------- ------ ... ------------------- --------------------
    6.8674186332946 6563.5 ... -12.793136209085365 0.003239857612426906
```

Better! By loosening the condition we added to the mean value, we got a better fit and the mean of the gaussian is closer to where we want it.

Exercise

Modify the value of delta to change the minimum and maximum values for the mean of the gaussian. Look for:

- The better delta so the mean is closer to the real value of the $H\alpha$ line.
- What is the minimum delta for which the fit is still good according to the plot?

What should you do if you need a model that `astropy.modeling` doesn't provide? To solve that problem, Astropy has another tool called custom model. Using this tool, we can create any model we want.

We will describe two ways to create a custom model:

- basic
- full

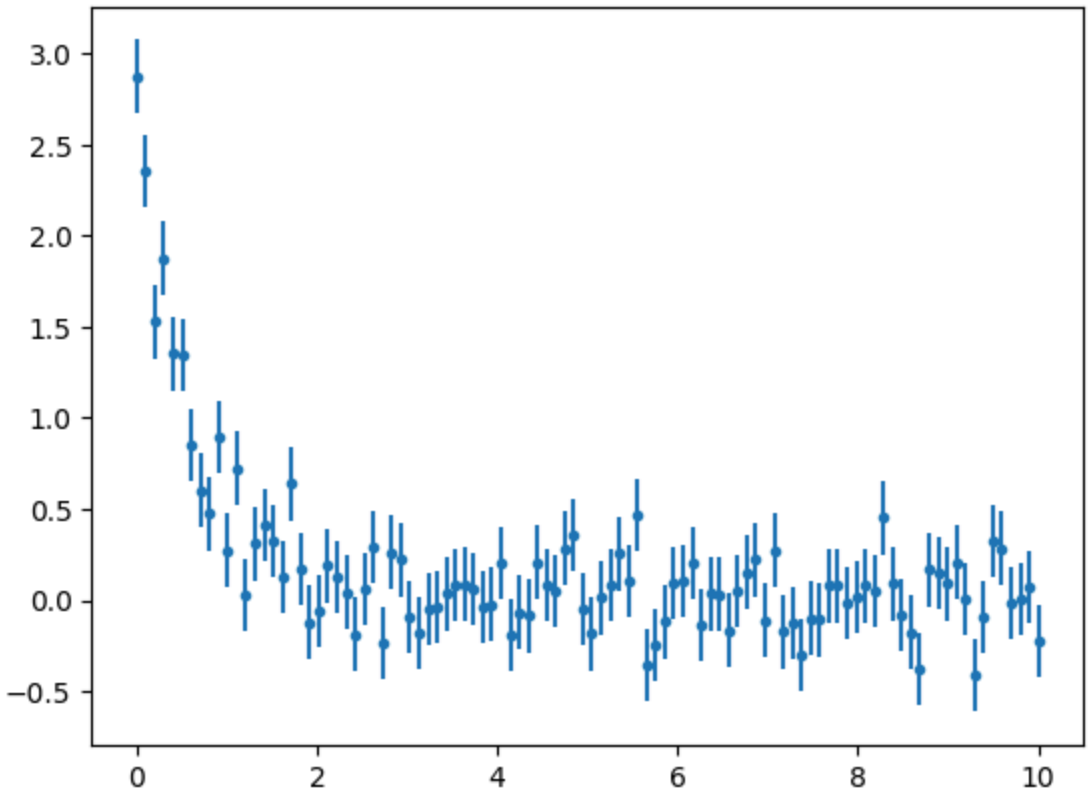
We use the basic custom model when we need a simple function to fit and the full custom model when we need a more complex function. Let's use an example to understand each one of the custom models.

Basic custom model

An **Exponential Model** is not provided by Astropy models. Let's see one example of basic custom model for this case. First, let's simulate a dataset that follows an exponential:

```
x1 = np.linspace(0, 10, 100)

a = 3
b = -2
c = 0
y1 = a * np.exp(b * x1 + c)
y1 += np.random.normal(0.0, 0.2, x1.shape)
y1_err = np.ones(x1.shape) * 0.2
```

```
plt.errorbar(x1, y1, yerr=y1_err, fmt=".")
plt.show()
```



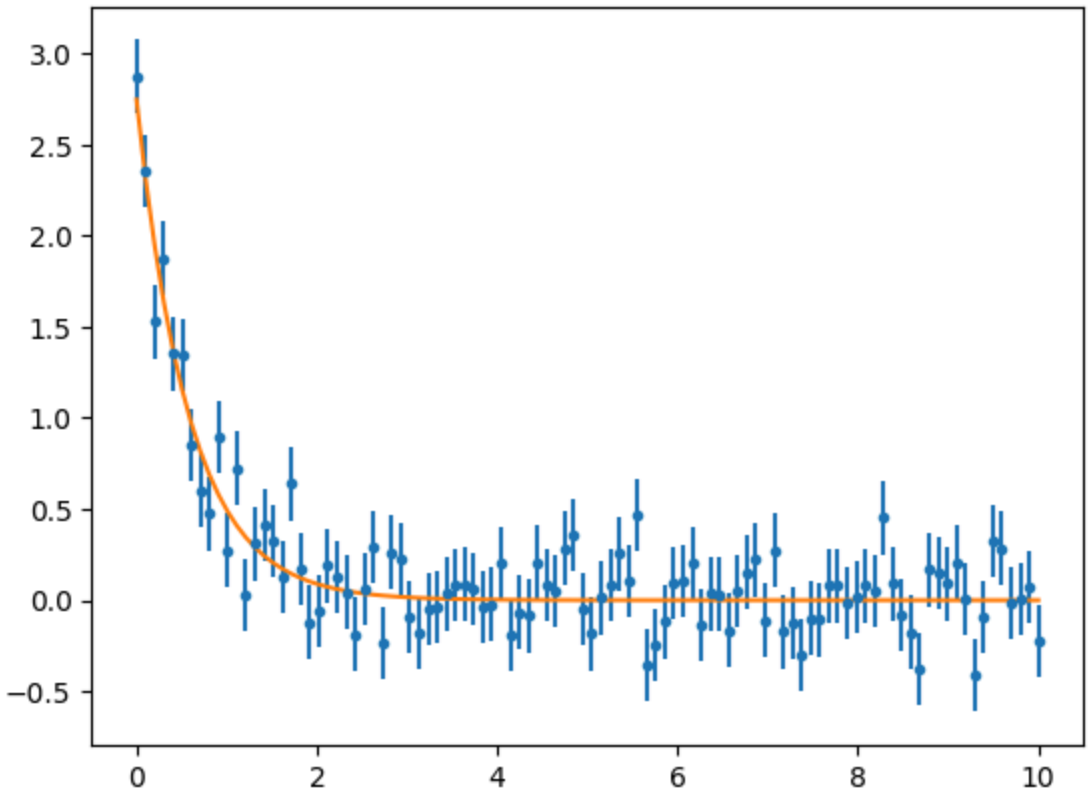
We can define a simple custom model by specifying which parameters we want to fit.

```
@custom_model
def exponential(x, a=1.0, b=1.0, c=1.0):
    """
    f(x)=a*exp(b*x + c)
    """
    return a * np.exp(b * x + c)
```

Now we have one more available model to use in the same way we fit data with `astropy.modeling`.

```
exp_model = exponential(1.0, -1.0, 1.0)
fitter = fitting.LevMarLSQFitter()
exp_fit = fitter(exp_model, x1, y1, weights=1.0 / y1_err**2)
```

```
plt.errorbar(x1, y1, yerr=y1_err, fmt=".")
plt.plot(x1, exp_fit(x1))
plt.show()
```




```
print(exp_fit)
```

```
Model: exponential
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
            a                    b                    c
    ------------------ -------------------- ------------------
    1.2960574409925534 -1.7129634872382655 0.7506240119002233
```

The fit looks good in the plot. Let's check the parameters and the Reduced Chi Square value, which will give us information about the goodness of the fit.

```
def calc_reduced_chi_square(fit, x, y, yerr, N, n_free):
    """
    fit (array) values for the fit
    x,y,yerr (arrays) data
    N total number of points
    n_free number of parameters we are fitting
    """
    return 1.0 / (N - n_free) * sum(((fit - y) / yerr) ** 2)
```

```
calc_reduced_chi_square(exp_fit(x1), x1, y1, y1_err, len(x1), 3)
```

```
np.float64(0.962630962901822)
```

The Reduced Chi Square value is close to 1. Great! This means our fit is good, and we can corroborate it by comparing the values we got for the parameters and the ones we used to simulate the data.

Note: Fits of non-linear parameters (like in our example) are extremely dependent on initial conditions. Pay attention to the initial conditions you select.

Exercise

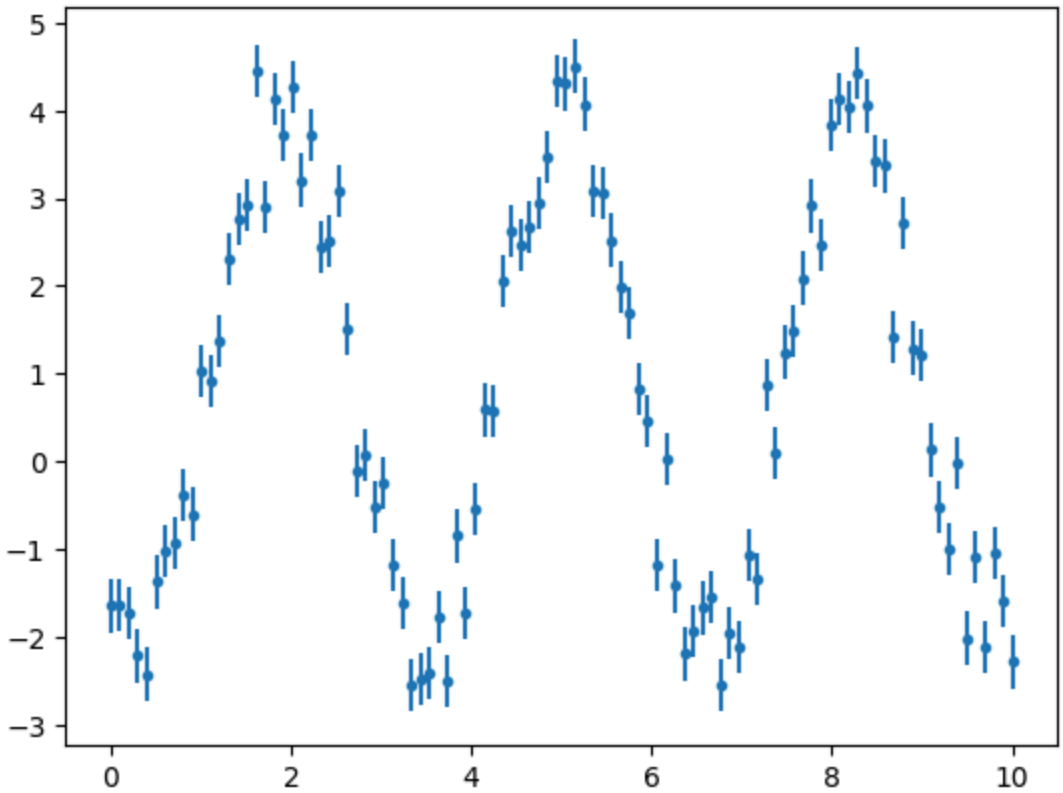
Modify the initial conditions of the fit and check yourself the relation between the best fit parameters and the initial conditions for the previous example. You can check it by looking at the Reduced Chi Square value: if it gets closer to 1 the fit is better and vice versa. To compare the quality of the fits you can take note of the Reduced Chi Square value you get for each initial condition.

Full custom model

What if we want to use a model from `astropy.modeling`, but with a different set of parameters? One example is the Sine Model. It has a very particular definition of the frequency and phase. Let's define a new Sine function with a full custom model. Again, first let's create a simulated dataset.

```
x2 = np.linspace(0, 10, 100)
a = 3
b = 2
c = 4
d = 1
y2 = a * np.sin(b * x2 + c) + d
y2 += np.random.normal(0.0, 0.5, x2.shape)
y2_err = np.ones(x2.shape) * 0.3
```

```
plt.errorbar(x2, y2, yerr=y2_err, fmt=".")
plt.show()
```



For the full custom model we can easily set the **derivative of the function**, which is used by different fitters, for example the `LevMarLSQFitter()`.

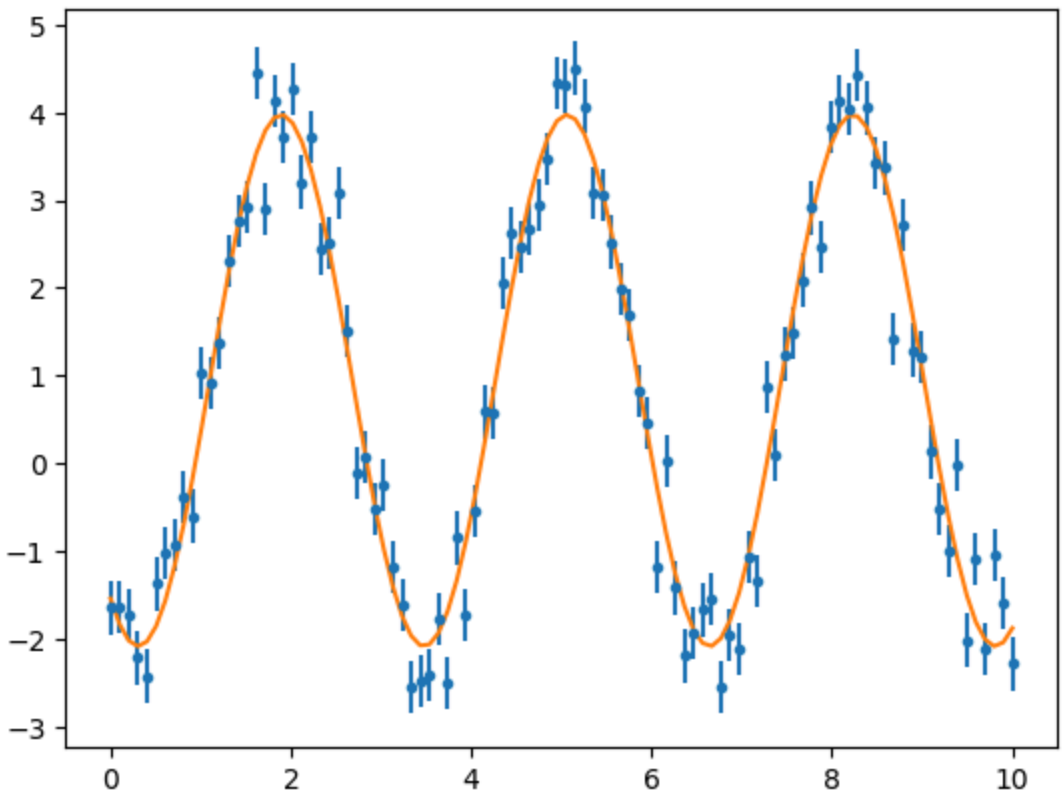
```
class SineNew(Fittable1DModel):
    a = Parameter(default=1.0)
    b = Parameter(default=1.0)
    c = Parameter(default=1.0)
    d = Parameter(default=1.0)

    @staticmethod
    def evaluate(x, a, b, c, d):
        return a * np.sin(b * x + c) + d

    @staticmethod
    def fit_deriv(x, a, b, c, d):
        d_a = np.sin(b * x + c)
        d_b = a * np.cos(b * x + c) * x
        d_c = a * np.cos(b * x + c)
        d_d = np.ones(x.shape)
        return [d_a, d_b, d_c, d_d]
```

Note Defining default values for the fit parameters allows to define a model as `model=SineNew()`

We are going to fit the data with our **new model**. Once more, the fit is very **sensitive to the initial conditions** due to the non-linearity of the parameters.

```
sine_model = SineNew(a=4.0, b=2.0, c=4.0, d=0.0)
fitter = fitting.LevMarLSQFitter()
sine_fit = fitter(sine_model, x2, y2, weights=1.0 / y2_err**2)
```

```
plt.errorbar(x2, y2, yerr=y2_err, fmt=".")
plt.plot(x2, sine_fit(x2))
plt.show()
```




```
print(sine_fit)
```

```
Model: SineNew
Inputs: ('x',)
Outputs: ('y',)
Model set size: 1
Parameters:
            a                  b                  c                  d
    ----------------- ------------------ ----------------- ------------------
    3.029556468904174 1.9831478983057012 4.102563083919028 0.9455117220244017
```

```
calc_reduced_chi_square(sine_fit(x2), x2, y2, y2_err, len(x2), 3)
```

```
np.float64(2.7218055563094414)
```

The Reduced Chi Squared value is showing the same as the plot: this fit could be improved. The Reduced Chi Squared is not close to 1 and the fit is off by small phase.

Exercise

Play with the initial values for the last fit and improve the Reduced Chi Squared value.

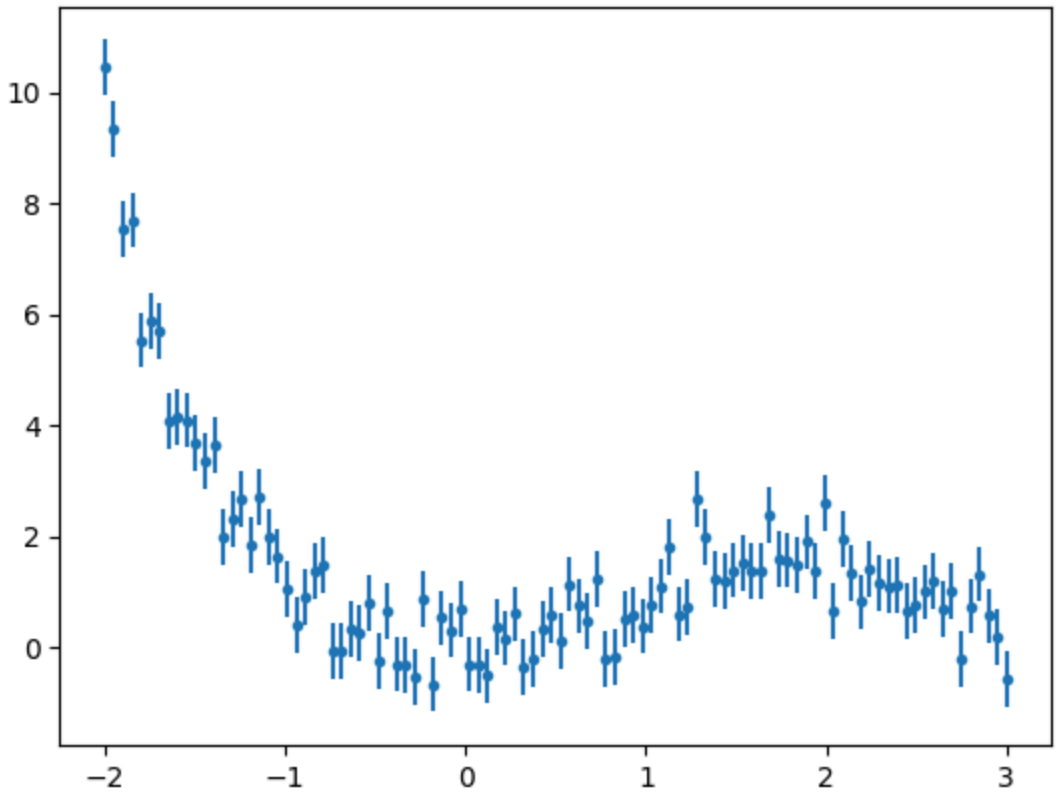
Note: A fancy way of doing this would be to code a function which iterates over different initial conditions, optimizing the Reduced Chi Squared value. No need to do it here, but feel free to try.

Exercise

Custom models are also useful when we want to fit an **unusual function** to our data. As an example, create a full custom model to fit the following data.

```
x3 = np.linspace(-2, 3, 100)
y3 = x3**2 * np.exp(-0.5 * (x3) ** 3 / 2**2)
y3 += np.random.normal(0.0, 0.5, x3.shape)
y3_err = np.ones(x3.shape) * 0.5
```

```
plt.errorbar(x3, y3, yerr=y3_err, fmt=".")
plt.show()
```


10
8
6
4
2
0
−2
−1
0
1
2
3